

iSHIFT : Lightweight Slow-Fast GUI Agent with Adaptive Perception

Sarthak Mehrotra^{1,3} Sairam VC Rebbapragada²
Mani Hemanth Reddy Bonthu² Vineeth N Balasubramanian^{2,4}

¹Indian Institute of Technology Bombay ²Indian Institute of Technology Hyderabad ³MDSR Lab Adobe ⁴Microsoft Research
sarthak2002.mehrotra@gmail.com {ai20resch13001, cs22btech11013, vineethnb}@iith.ac.in
vineeth.nb@microsoft.com

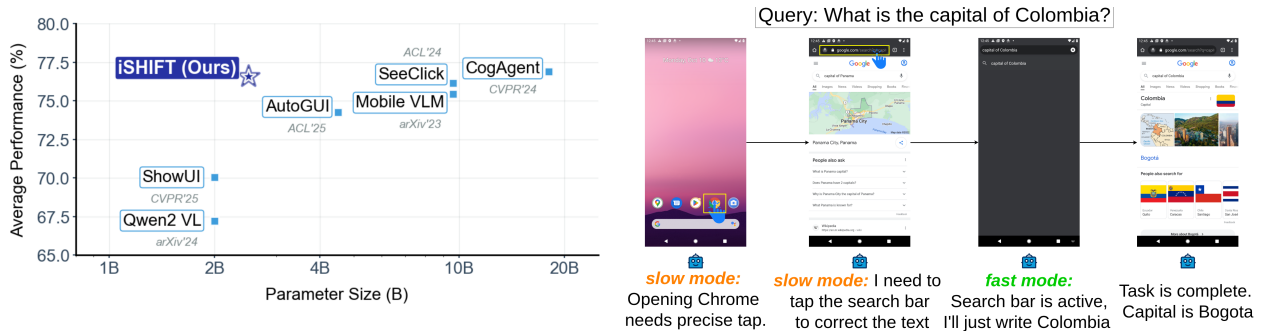


Figure 1. **Left:** Compared to recent state-of-the-art methods on Android In The Wild Benchmark [23], our approach achieves a competitive performance despite smaller size. **Right:** iSHIFT achieves this by adaptively switching between slow and fast modes based on task needs.

Abstract

Multimodal Large Language Models (MLLMs) show strong potential for interpreting and interacting with complex, pixel-rich Graphical User Interface (GUI) environments. However, building agents that are both efficient for high-level tasks and precise for fine-grained interactions remains challenging. GUI agents must perform routine actions efficiently while also handling tasks that demand exact visual grounding, yet existing approaches struggle when accuracy depends on identifying specific interface elements. These MLLMs also remain large and cannot adapt their reasoning depth to the task at hand. In this work, we introduce **iSHIFT: Implicit Slow-fast Hybrid Inference with Flexible Tokens**, a lightweight agent that integrates latent thinking (implicit chain-of-thought) with a perception control module. **iSHIFT** enables an MLLM to switch between a slow mode, which leverages detailed visual grounding for high precision and a fast mode that uses global cues for efficiency. Special perception tokens guide attention to relevant screen regions, allowing the model to decide both how to reason and where to focus. Despite its compact **2.5B** size, **iSHIFT** matches state-of-the-art performance on multiple benchmark datasets. Code is available [here](#).

1. Introduction

Agents capable of understanding and operating Graphical User Interfaces (GUIs) are emerging as a pragmatic approach for automating everyday digital tasks. Acting as virtual intermediaries, such agents can mitigate user effort, orchestrate complex workflows, and enhance accessibility across mobile, desktop, and web platforms [14, 15, 23, 24]. Initial research efforts converged on language-driven pipelines contingent upon *structural oracles* such as HTML, Document Object Model (DOM), or accessibility trees (AXTree) [8, 42]. While effective under ideal conditions, these methodologies suffer from poor generalization, as they are often tied to platform-specific inconsistencies and require complex preprocessing heuristics [18, 21]. Moreover, the frequent unavailability or the need for special permissions to access this structural information at runtime curtails their practical deployment.

These constraints prompted a shift towards vision-based and vision-language agents that operate directly upon screenshot-derived pixel data, mirroring human perceptual input, to generate grounded actions [5, 16, 35]. Nevertheless, engineering reliable agents from raw pixels remains a challenge. GUIs contain fine-grained text, icons, and interactive elements within highly variable layouts, demanding both meticulous perception and robust contextual reason-

ing. Many existing approaches mitigate this by employing resource-intensive perception stacks, incorporating Optical Character Recognition (OCR), segmentation, or specialized backbones to enhance accuracy. This strategy, however, imposes a significant computational burden, as heavy perception modules remain constantly active, incurring undue latency and cost even for trivial actions [10, 22, 26, 37].

Additionally, a central limitation pervading the GUI agents is their absence of *adaptive* resource allocation. While slow-fast paradigms are studied in general-purpose visual agents [6, 27], their application to GUI navigation remains nascent. Consequently, most GUI agents process all actions uniformly, disregarding the natural dichotomy within UI interactions between *fast actions* (e.g., swiping) executable with global context, and *slow actions* (e.g., finding a small button) that necessitate deliberate reasoning or enhanced visual precision. This uniform treatment compromises the balance between efficiency and reliability: models either expend excessive computation on simple tasks or fail to allocate sufficient reasoning for difficult ones. A recent work [28] attempts to make GUI automation adaptive, however, it relies on generating *explicit intermediate representations* (e.g., icon captions) to improve grounding. For complex tasks, its slow path activates a text-generation loop, forcing the model to generate interface summaries and visual analyses before it can act. This approach incurs generative latency and couples grounding accuracy to the quality of the generated text. This multi-stage process trades one set of problems for another and highlights the need for a more integrated solution.

To address these limitations, we introduce **iSHIFT** (Implicit Slow-fast Hybrid Inference with Flexible Tokens), a $\sim 2.5\text{B}$ -parameter multimodal GUI agent that natively integrates both compute and perception adaptivity within a unified model. iSHIFT is founded on the principle that the model itself should *implicitly* determine when to engage in deeper, *slow* reasoning. It achieves this through two complementary mechanisms: **latent thinking tokens**, which facilitate non-verbal internal deliberation, and a **perception token**, which on-demand activates a lightweight, localized visual perception module. This co-design obviates the need for external controllers, tightly couples reasoning depth with selective visual acuity, and achieves both efficiency on simple interactions and precision on complex ones. As depicted in Figure 1, our framework enables a compact agent to achieve state-of-the-art performance-to-size trade-off (Fig. 1 left), by adaptively switching between *fast* and *slow* reasoning modes during action episodes (Fig. 1 right).

Our main contributions are as follows:

1. We introduce **iSHIFT**, a unified multimodal agent that integrates intrinsic slow-fast control. It leverages latent thinking tokens to **implicitly** and **on-demand** trigger a

lightweight visual perception module, thereby eliminating external controllers and intermediate representations.

2. We propose a novel training strategy for instilling adaptive control. By programmatically annotating training data to conditionally include perception tokens for high-precision tasks, the model learns to correlate task complexity with its implicit slow-fast mechanisms.
3. We demonstrate that our agent achieves state-of-the-art performance-to-compute trade-off. Across multiple benchmarks, iSHIFT either surpasses or matches substantially larger models in performance while utilizing much fewer parameters.

2. Related Work

Vision-Based GUI Agent Architectures. Early GUI automation efforts often relied on structured interface representations like Document Object Model (DOM) trees or platform-specific accessibility data [1, 8]. While effective in constrained environments like web pages, these methods lack generality, as such structural information is often unavailable or inconsistent across arbitrary desktop and mobile applications. The rise of powerful vision-language models (VLMs) has enabled agents that operate directly from pixel-level observations, closely mirroring human perception [5, 7, 10, 11, 13, 22, 25, 35, 40]. To improve UI understanding, [5] introduces grounding-based pretraining, while subsequent work highlights a core challenge: GUIs are high-resolution and densely populated with small but crucial elements (e.g., icons, text fields) that are easily lost under naive downsampling. To preserve fine-grained details, several methods employ high-resolution encoders [10, 22], improving grounding at the cost of significant computational overhead. [11] addresses locational ambiguity in dynamic high-resolution interfaces through Universal Block Parsing (UBP), enhancing GUI element localization. Meanwhile, [13] mitigates token inefficiency by introducing UI-guided visual token selection, grouping visually similar regions to reduce the processing burden.

Slow-Fast Approaches. To mitigate the trade-off between efficiency and fidelity, a *slow-fast* paradigm has emerged. These methods explicitly acknowledge that not all tasks require the same level of computation. However, existing implementations introduce new architectural complexities that undermine their benefits. For example, [27] proposes a system that arbitrates between a *fast* and *slow* path, but it relies on an external controller to make this decision. This disjointed design introduces modular complexity, adds latency, and creates a new potential point of failure. Similarly, GUI-specific work like [28] improves precision by generating explicit intermediate representations (e.g., icon captions) as a prerequisite slow grounding step. This approach, while effective, exacerbates system complexity and inference time by forcing the model to generate verbose text before it can

act. Both methods validate the need for adaptive computation but highlight a research gap: the lack of a unified and implicit control mechanism.

Thought-Based GUI Agents. Another line of research enhances agent reliability by incorporating explicit cognitive loops, often inspired by Chain-of-Thought (CoT) reasoning [32]. Agents such as [30] adopt prompt frameworks like ReAct [36], interleaving reasoning steps with actions to form explicit plans and reflect on intermediate observations. [33] strengthens action prediction by generating thoughts that analyze instructions, screenshots, and interaction history. Similarly, [34] leverages structured reasoning to decompose complex tasks, adapt to new scenarios, and plan actions more effectively. However, a key limitation of these approaches is their dependence on explicit natural language generation. This verbosity incurs substantial inference latency, which is problematic for interactive GUI tasks where rapid responses are essential.

3. iSHIFT: Methodology

We now describe our iSHIFT methodology, which is centered on the core principles of implicit thinking and adaptive resource allocation. We begin by explaining the primary contribution: the iSHIFT Slow-Fast Approach (§3.1), which forms the operational core of our agent. This mechanism allows the model to implicitly shift between a computationally efficient *Fast Path* for simple actions and a high-precision *Slow Path* for complex ones. We then detail the two key Architectural Components that enable this adaptability: the *Latent Thinking Tokens* for general deliberation, and the *Visual Perception Module* which provides the localized features for the slow path.

3.1. The iSHIFT Slow-Fast Approach

The central innovation of iSHIFT is an adaptive processing mechanism that addresses the dichotomy in UI tasks: simple actions (e.g., swipe left) can be executed with global context, while complex actions (e.g., tap tiny icon) demand precise, localized visual processing. Our model addresses this by following a sequential process detailed in Algorithm 1. Our process uses a *Fast Path* by default and allows the model to *conditionally* activate a *Slow Path* for added precision *only when necessary*. We briefly describe our process herein, and then detail the components later in this section.

Step 1: Implicit Thinking (*Fast Path*). As shown in Figure 2, our model always begins with the computationally efficient *Fast Path*, which processes a given user instruction and a UI screenshot. The model then performs implicit deliberation using the *Latent Thinking Tokens* (<bot>...<eot>), inspired by [9], that are detailed in §3.2. This step serves as the model’s internal assessment phase, where it evaluates the task’s requirements based on the given context.

Step 2: The Implicit Shift Decision. Following the initial

deliberation, the model makes a critical decision implicitly: “Is the current information sufficient to act?” This decision is not made by an external controller, but via tokenization on the model’s next generative step.

- *Stay on Fast Path:* If the deliberation concludes that the task is simple (e.g., a swipe), the model proceeds to generate the action directly, bypassing step 3.
- *Activate Slow Path:* If the deliberation concludes that the task requires high precision (e.g., click on a small icon), the model instead generates special *Latent Perception Tokens*: <bop>, <ctrl>, <eop>, similar to [38], to invoke the *Visual Perception Module* detailed in §3.2.

Step 3 (Optional): Conditional Activation (*Slow Path*). The generation of the <bop> token provides an explicit signal that activates the *Slow Path*. This is not a restart but rather a conditional detour for feature enhancement. This token triggers the *Visual Perception Module*, which uses a DINO [20] encoder to extract localized, fine-grained features z_p from the screen. These new features are then injected back into the model’s sequence.

Step 4: Final Action Generation. The model, now armed with the initial global features and optionally the augmented, fine-grained features (from the *Slow Path*) generates the action.

Training the Implicit Shift. This entire conditional process is learned end-to-end. We teach the model this ability through a dataset enhancement step, where we programmatically categorize the action space of our datasets based on their perception requirements to include our *latent thinking tokens* and *latent perception tokens*, as required. This enables the model to learn a direct association between the contextual scenario and the correct execution path. Any action that requires the prediction of precise coordinates (e.g., clicking on a UI element) is labeled as a *Slow Action*. Prompt labels for such actions are annotated to include the *latent perception tokens* (<bop>, <ctrl> and <eop>). Observing this token sequence prompts the model to engage the *Visual Perception Module* to acquire additional context required for more precise grounding. All other actions that can be inferred from global context (e.g., swiping) are labeled as *Fast Actions*. They are modeled to follow the *Fast Path*, bypassing the *Visual Perception Module*. This mapping provides the necessary supervision to enable the model to learn the association between task types (inferred from the instruction and screen) and the capability to generate the <bop> token to activate the *Slow Path*.

In contrast to prior work which use an explicit external controller, our iSHIFT framework embeds this critical decision-making step implicitly within our unified MLLM architecture. As shown in Figure 2, this implicit mechanism allows the model to perform internal, non-linguistic deliberation and decide whether sufficient context exists or

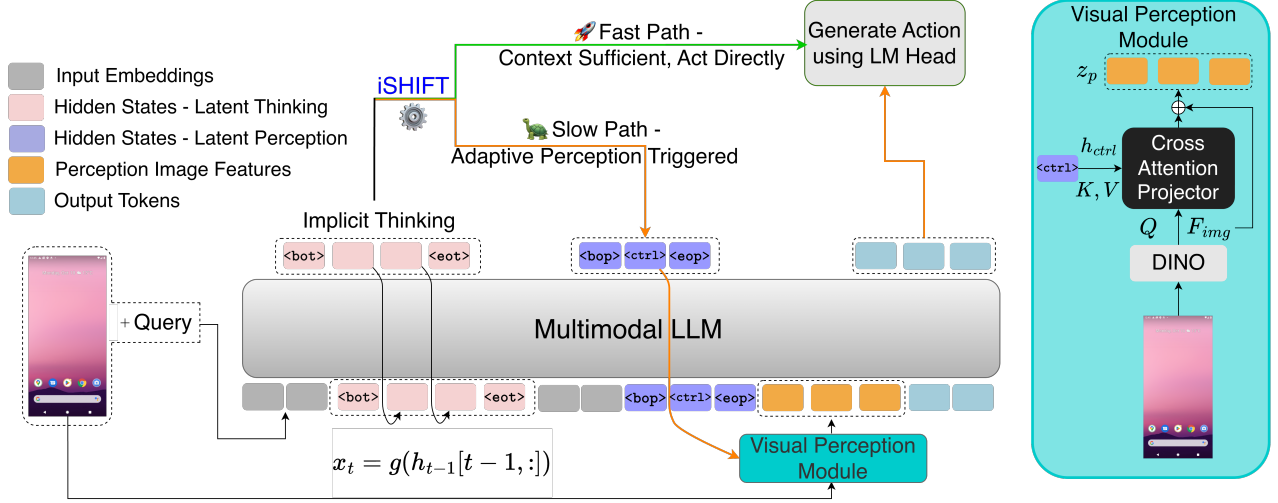


Figure 2. **Overview of our proposed method.** The MLLM takes a screenshot and task query as input and begins with the **Fast Path**. It utilizes the **Latent Thinking Tokens** (<bot>...<eot>) for implicit thinking to assess if the context is sufficient for direct action. If not, it switches to the **Slow Path** by generating **Latent Perception Tokens** (<bop>, <ctrl>, <eop>). This invokes the **Visual Perception Module** which extracts localized image features (z_p) for precise and grounded action generation.

if additional visual information is required. This eliminates reliance on external components, keeping the model’s rationale embedded in its hidden states and providing it with context-awareness.

Algorithm 1 Slow–Fast Adaptive Control in iSHIFT

Require: Input context $C = \{I, T\}$, where I is the UI screenshot and T is the textual instruction

Ensure: Predicted action A

```

1: Initialize token sequence  $\mathcal{S} \leftarrow [\text{<bot>}]$ 
2: Initialize latent hidden state  $h_0$ 
3: while not end of sequence do
4:    $h_t \leftarrow \text{Transformer}(h_{t-1}, \mathcal{S}_t)$ 
5:
6:   while  $t$  within <bot> and <eot> do
7:      $x_t \leftarrow g(h_{t-1}[t-1, :])$ 
8:     Model performs implicit reasoning over context
9:   end while
10:  Predict internal control decision  $d_t \in \{\text{Fast}, \text{Slow}\}$ 
11:  if  $d_t = \text{Slow}$  then
12:    Generate latent control sequence:
        
$$\mathcal{S}_{ctrl} = \{\text{<bop>, <ctrl>, <eop>}\}$$

13:    Add latent control sequence  $\mathcal{S} \leftarrow \mathcal{S}_{ctrl}$ 
14:    Extract control state  $h_{ctrl} \leftarrow h_t[\text{<ctrl>}]$ 
15:     $F_{img} \leftarrow \text{DINOEncoder}(I)$ 
16:     $z_p \leftarrow \text{CrossAttention}(F_{img}, h_{ctrl})$ 
17:     $\mathcal{S} \leftarrow \mathcal{S} + z_p$ 
18:  end if
19: end while
20: Generate final action  $A \leftarrow \text{LM}(h_T)$ 

```

3.2. Architecture Components

The Slow-Fast approach is enabled by two key architectural additions to the Qwen2-VL 2B [31] base model.

Latent Thinking Tokens. We integrate a continuous latent thinking mechanism, to enable the model to perform

introspection without generating explicit language tokens. As mentioned in §3.1, this mechanism introduces special tokens, <bot> and <eot>, which indicate the beginning and end of a continuous thought block. Using these tokens, the model develops its reasoning in the latent space, without decoding intermediate language outputs.

This operation transforms the hidden state directly into the next input embedding, bypassing the vocabulary layer entirely. By operating fully within this latent space, the model achieves a deeper representational flow that captures implicit reasoning with significantly fewer steps compared to explicit Chain-of-Thought (CoT) generation. The resulting latent thinking process governs the model’s internal deliberation and is responsible for triggering the **Slow Path** when extended reasoning is required, ensuring resource-efficient yet contextually adaptive action generation.

The model learns to use these latent tokens via a specific training methodology. Initially, the model is trained on a smaller dataset where the thought token positions (<bot>...<eot>) are populated with explicit thoughts in natural language. This phase is crucial because it teaches the model the correct generation process and the expected content of its internal deliberations. Once this foundation is learned, the explicit thought tokens are replaced with the final latent thought tokens, giving the model a strong prior on the latent structure and the type of information it should process internally. Since thoughts are ultimately represented latently rather than in natural language, no additional thought annotations are required when scaling to larger datasets ($\approx 50\times$) improving overall scalability. More details about the training and implementation are presented in the Supplementary.

Method	Base Model	Size	Vision Only	General	Install	Google Apps	Single	WebShop	Avg.
Models with parameter count above 5B									
LLaMa 2 [29] (arxiv'23)	LLaMa 2	7B	✗	28.56	35.18	30.99	27.35	19.92	28.40
CoCo-Agent [17] (arxiv'24)	LLaMa 2	7B	✗	69.92	80.60	75.76	88.81	74.02	77.82
MobileAgent [30] (ICLR'24 w)	LLaMa	7B	✗	55.80	74.98	63.95	76.27	63.61	66.92
GPT-4o [19] (arxiv'24)	–	–	✓	47.06	49.12	52.30	80.28	46.42	55.02
OmniParser [16] (arxiv'24)	–	–	✓	48.30	57.80	51.60	77.40	52.90	57.60
CogAgent [10] (CVPR'24)	Vicuna 7B	18B	✓	65.38	78.86	74.95	<u>93.49</u>	71.73	<u>76.88</u>
SeeClick [5] (ACL'24)	QwenVL	9.6B	✓	67.60	79.60	<u>75.90</u>	84.60	73.10	76.16
Mobile VLM [7] (EMNLP'24)	QwenVL	9.6B	✓	69.58	79.87	74.72	81.24	71.70	75.42
SphAgent [3] (ACL'25)	InternLM	7B	✓	68.20	80.50	73.30	85.40	<u>74.00</u>	76.28
TongUI [40] (arxiv'25)	Qwen2.5 VL	7B	✓	67.6	76.3	73.5	79.9	69.1	73.3
Models with parameter count below 5B									
AutoGUI [41] (ACL'24)	T5 + ViT	4.5B	✓	68.24	76.89	71.37	84.58	70.26	74.27
Qwen2 VL [31] (arxiv'24)	Qwen2 VL	2B	✓	61.40	71.80	62.60	73.70	66.70	67.24
ShowUI [13] (CVPR'25)	Qwen2 VL	2B	✓	63.90	72.50	69.70	77.50	66.60	70.04
TongUI [40] (arxiv'25)	Qwen2.5 VL	3B	✓	65.6	75.10	74.50	77.00	65.8	71.6
iSHIFT	Qwen2 VL	2.5B	✓	70.6	80.82	71.64	86.03	72.60	76.34

Table 1. Evaluation results across various subsets on the Android In The Wild [23] Benchmark, measured by Action Matching Score. Best scores in the category < 5B are **bolded**. Overall best scores are underlined. Cells in the iSHIFT row with a Green fill demonstrate **SOTA performance parity**, meaning the score is within 2.0 points of the column’s maximum value.

Visual Perception Module. The **Visual Perception Module** serves as the engine for the **Slow Path**. When activated by the <bop> token, the image is re-encoded into information-rich feature maps (F_{img}) using a DINO [20] vision encoder. These visual features then cross attend with the **latent perception token’s** hidden states (h_{ctrl}) through a cross-attention projector. This allows the model to selectively attend to and aggregate critical visual details.

The new context embedding z_p is derived via cross-attention where the image features act as the Query (Q) and the **latent perception token’s** hidden state act as the Key and Value (K, V):

$$Q = \text{Linear}_Q(F_{img}); K, V = \text{Linear}_{K,V}(h_{ctrl})$$

$$z_p = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V + F_{img}$$

where d_k is the dimensionality of the query vector Q .

The resulting context embedding z_p is injected back into the MLLM sequence, providing the fine-grained control needed for precise grounding. Crucially, unlike methods such as CogAgent [10] or V-Zen [22], our **Visual Perception Module** is lighter and activated only when necessary. This allows iSHIFT to achieve superior precision without the large increase in parameter count that plagues high-resolution modules in existing models.

4. Experiments

Datasets, Baselines and Evaluation Metrics. We evaluate iSHIFT across diverse GUI benchmarks including Android In The Wild [23] (AITW), Android Control [12],

GUIOdyssey [15], and GUIAct [4]. For comparison, we use state-of-the-art GUI agents such as [5, 10, 12, 13, 40] which represent various approaches to high-performance and high-resolution GUI processing. Our evaluation reports the Action Matching Score (AMS) for AITW (across General, Single, Install, Web Shopping, Google Apps subsets) and Android Control (High and Low subsets). For GUIOdyssey, we report the Success Rate, and for GUIAct (Web Single and Phone subsets), we provide both Type accuracy and Success Rate metrics. More details related to the above are mentioned in the Supplementary.

Results and Observations. The iSHIFT model, built on a compact 2.5B parameter base, achieves an Average Action Matching Score (AMS) of **76.34%**, establishing it as the SOTA leader among models under 5B parameters and reaching near parity with the overall SOTA model, CogAgent [10] (18B), with only a 0.54 point difference (76.34% vs. 76.88%) despite being more than seven times smaller. Relative to its closest competitor in the small model category, AutoGUI [39] (4.5B), iSHIFT provides a gain of over 2.0 points (76.34% vs. 74.27%) while remaining significantly smaller in size. Its adaptive processing capability is consistently reflected across individual subsets: iSHIFT sets new SOTA performance on the General and Install subsets with AMS scores of **70.6%** and **80.82%**, respectively, and achieves the highest score in the Single subset within the < 5B category (**86.03%**), highlighting the effectiveness of its adaptive Perception Tokens for precise and efficient action grounding. On WebShop, it continues to lead its category with **72.60%**, demonstrating strong composi-

tional reasoning in structured environments, and it performs competitively on Google Apps with 71.64%, maintaining robust generalization across diverse UI layouts and interaction types.

The superior performance-to-size ratio of iSHIFT is clearly visualized in Figure 4, which plots the Efficiency Metric (Accuracy/Parameter Count) for iSHIFT compared to different SOTA models in the respective subsets. iSHIFT significantly outperforms all other SOTA models, achieving an efficiency of **30.54** on Average, which is more than five times that of the 18B CogAgent [10] (which scores ~ 4.27 on Average). iSHIFT achieves a score of **28.65** in Google Apps, approximately four times that of the 9.6B SeeClick [5] (which scores ~ 7.9). This provides concrete evidence that our adaptive token mechanism allows a highly compact 2.5B MLLM to achieve results comparable to substantially larger agents, setting a new standard for computational efficiency in resource-aware GUI agents.

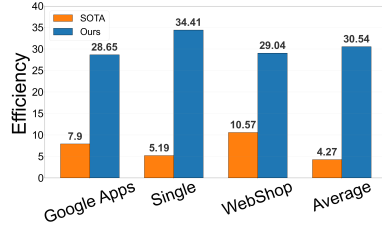


Figure 4. Efficiency comparison (Accuracy-to-Parameter ratio) of iSHIFT against subset wise state-of-the-art agents.

The evaluation of iSHIFT on cross-device benchmarks such as GUI Odyssey, summarized in Table 2, demonstrates strong generalization despite its compact 2.5B size, achieving the highest overall success rate of **73.97%** across subsets and outperforming Aguis (63.8%), which is about four times larger. iSHIFT also secures the top score in the Android Control (Low) subset with **87.7%** and maintains competitive performance in the High-level precision subset with 65.6%, ranking just below OS Atlas (4B), which achieves 67.54% in the same category. As shown in Table 3, iSHIFT continues to excel on the Web Single subset of GUIAct with a Type Accuracy of **93.83%**, surpassing GUICourse 3.1B (91.8%), while retaining a strong Success Rate of 66.38%. On the Phone subset, it attains **79.41%** in Type Accuracy and **60.08%** in Success Rate, outperforming the much larger GUICourse 9.6B model, which records 73.0% and 58.1%, respectively. These results highlight iSHIFT’s consistent high-fidelity performance across platforms while maintaining exceptional parameter efficiency.

Qualitative Results We present a qualitative comparison of GUI action sequences by evaluating iSHIFT, SeeClick and ShowUI against Ground Truth, in Figure 3. More qualitative results can be found in the Supplementary.

5. Analysis and Ablation Studies

To better understand the effectiveness of our approach, we systematically evaluate different components of the pro-

Method	Size	Android Control		GUI
		High	Low	Odyssey
Models with parameter count above 5B				
GPT-4o [19] (arxiv’24)	–	28.39	21.17	5.36
See Click [5] (ACL’24)	9.6B	59.1	75.0	53.92
OS Atlas [33] (ICLR’25)	7B	<u>71.17</u>	85.22	61.98
TongUI [40] (arxiv’25)	7B	38.2	69.3	–
Aguvis [34] (ICML’25)	7B	61.5	80.5	63.8
Models with parameter count below 5B				
Qwen2.5-VL [2] (arxiv’25)	3B	58.79	41.22	27.31
OS Atlas[33] (ICLR’25)	4B	67.54	80.64	56.39
TongUI [40] (arxiv’25)	3B	32.0	52.0	–
iSHIFT	2.5B	65.6	<u>87.7</u>	<u>73.97</u>

Table 2. Evaluation results on Android Control [12] and GUI Odyssey [15] benchmarks. Best scores in the category < 5B are **bolded**. Overall best scores are underlined.

Method	Size	Web Single		Phone	
		Type	SR	Type	SR
Models with parameter count above 5B					
GPT-4o [19] (arxiv’24)	–	77.09	41.84	46.9	28.4
GUICourse [4] (ACL’25)	9.4B	90.1	63.5	72.1	40.4
GUICourse [4] (ACL’25)	9.6B	90.9	66.7	73	58.1
OS-Atlas [33] (ICLR’25)	7B	86.95	57.02	–	–
Models with parameter count below 5B					
Qwen2.5-VL [2] (arxiv’25)	3B	56.1	55.61	60.8	35.8
GUICourse [4] (ACL’25)	3.1B	91.8	70.6	71.7	53.3
OS-Atlas [33] (ICLR’25)	4B	79.22	42.62	–	–
iSHIFT	2.5B	93.83	66.38	79.41	60.08

Table 3. Evaluation results on Web Single and Phone subset of GUIAct [4] benchmark. Overall best scores are **bolded**.

Variation	VPM	LTT	Slow	Fast	AMS
Base Model (Qwen2 VL 2B)	×	×	×	✓	67.24
w/ Fast Path Only	×	✓	×	✓	72.54
w/ VPM, Slow Path Only	✓	×	✓	×	72.71
w/ VPM, Adaptive	✓	×	✓	✓	75.48
w/ VPM + LTT, Slow Path Only	✓	✓	✓	×	74.58
w/o Cross-Attention in VPM	✓	✓	✓	✓	73.40
iSHIFT (Ours)	✓	✓	✓	✓	76.34

Table 4. Ablation study on the AITW benchmark [23] evaluating different components of iSHIFT. VPM denotes the Visual Perception Module and LTT refers to Latent Thinking Tokens.

posed framework as presented in Table 4. We evaluate how each module and design choice contributes to the overall performance on AITW [23]. All other settings remain consistent with the full model unless stated otherwise.

Impact of Individual Components. The standard Qwen2-VL-2B Baseline achieved an AMS of 67.24 on AITW [23] benchmark (Table 4). Adding Latent Thinking Tokens (LTT Only) boosts performance by +5.30 to 72.54 (Row 2, Table 4), validating the intrinsic benefit of implicit de-

QUERY: Open the contacts



Figure 3. Qualitative comparison of GUI action sequences. The figure shows step-by-step actions taken by SeeClick (red), ShowUI (orange), and iSHIFT (blue) in comparison to Ground Truth (green). Each colored hand or arrow represents the predicted interaction, including touch and lift points across interface states, and the ROI is highlighted with a yellow bounding box at each step. iSHIFT aligns closely with the correct action sequence, demonstrating superior spatial precision and decision accuracy compared to prior methods.

Method	AITW General		AITW Single	
	Runtime (ms)	AMS	Runtime (ms)	AMS
Fast	2093	66.64	2046	82.55
Slow	2331	68.2	2323	85.33
OURS	2229	70.6	2263	86.03

Table 5. Comparison of performance and latency between Adaptive iSHIFT and its fixed-path variants, showing that the adaptive version delivers higher accuracy with lower runtime.

liberation for general task execution. Similarly, introducing the Visual Perception Module (VPM) to the baseline yields a comparable +5.47 gain, reaching 72.71 (Row 3, Table 4). The largest improvement arises from enabling path selection. When both slow and fast paths are available, a non-LTT-based implicit decision process (Row 4, Table 4) achieves 75.48, a +2.77 increase over the slow-only variant (Row 3, Table 4), confirming the key role of path switching. Finally, the Full iSHIFT model, combining LTTs with real-time adaptive control of the VPM, reaches the best AMS of 76.34 (Row 7, Table 4). This LTT-driven decision-making contributes a further +0.86 beyond the non-LTT-based variant (Row 4, Table 4). Conversely, forcing iSHIFT to operate only on the slow path lowers performance to 74.58 (Row 5, Table 4), showing that true adaptive switching adds +1.76 and is central to iSHIFT’s superiority.

Why is Adaptive Better than Always Slow? While the

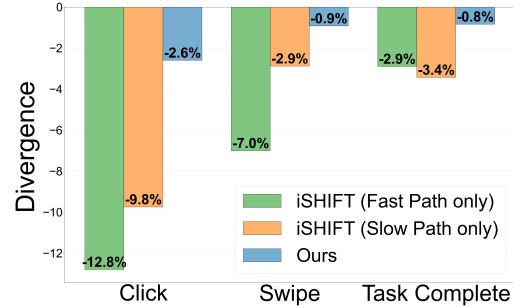


Figure 5. Divergence of distributions of predicted action from the Ground Truth across different iSHIFT variants (Smaller bar is better), This demonstrates the adaptive model’s superior action fidelity and significantly lower error rates across all actions.

iSHIFT (Slow Path Only) variant achieves a strong AMS of 74.58, it still lags behind the full adaptive model, suggesting that forced commitment to the slow path limits overall performance. As shown in Table 5, the adaptive iSHIFT model not only achieves higher AMS but also executes faster than the slow only variant, indicating that more computation time does not necessarily translate to better performance.

Figure 5 highlights that the adaptive model maintains a much smaller action type divergence from the Ground Truth across all key actions compared to the slow-only model. For example, in complex click actions, its deviation is only

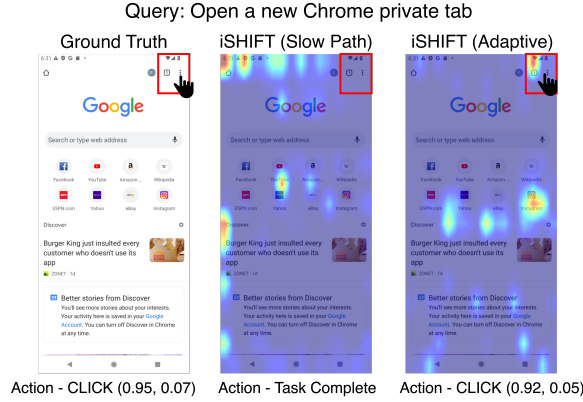


Figure 6. Attention maps from the cross-attention projector and the actions generated. This illustrates that mandatory localized image features in iSHIFT (Slow Path) produces diffused and irrelevant attention, leading to an incorrect action. In contrast, Adaptive iSHIFT (Ours) selectively attends to the correct target element, ensuring accurate action execution.

2.4% compared to 8.9% for the slow-only model. This discrepancy arises because the slow path variant processes detailed visual context indiscriminately for every action, even when such context is unnecessary. As illustrated in Figure 6, this leads to diffused and misdirected attention, where the model struggles to distinguish different actions, often misclassifying actions such as predicting Task Complete instead of Click. The continual, unnecessary addition of visual context for simple actions (e.g., Task Complete) hinders decision clarity and leads to suboptimal behavior. In contrast, the adaptive iSHIFT selectively retrieves context only when needed, maintaining sharper attention on the true target element and ensuring the correct action is executed. Together, these results confirm that iSHIFT’s implicit adaptive control enhances both accuracy and efficiency, eliminating unnecessary over-processing on simpler tasks.

Importance of Cross Attention Projector in VPM. Removing cross-attention prevents the model from accessing localized visual features, causing the Slow Path to fall back to coarse DINO features and rely only on global context. This weakens precise grounding and leads to a reduced AMS of 73.40 (Row 6, Table 4).

Effect of Number of Latent Tokens. We conducted an ablation to determine the optimal number of latent tokens for effective implicit deliberation, since an overly long thought block increases computation without proportional gains. As shown in Figure 7, the baseline without latent tokens reached an AMS of 75.48. Adding four latent tokens raised this to 75.72, and using eight tokens yielded the best performance at 76.34, providing sufficient reasoning depth with minimal overhead. Increasing the sequence further caused clear degradation, dropping to 74.09 with sixteen tokens and 74.04 with twenty. This non-linear trend shows that the length of the im-

plicit thought block is a sensitive hyperparameter, and excessive deliberation can dilute contextual focus and hinder efficient sequence generation, ultimately reducing performance.

We present more experiments related to Latent

Thinking Tokens in the Supplementary.

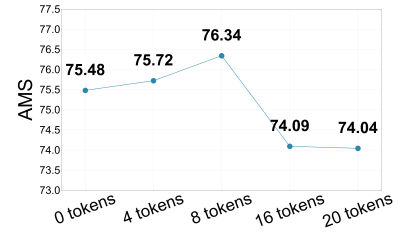


Figure 7. Impact of latent token count on average performance, showing optimal results around 8 tokens.

Effect of Different Encoders in VPM. We conducted an ablation on the visual encoder used in the Visual Perception Module (VPM) to evaluate performance and efficiency, as shown in Figure 8.

The standard Qwen2-VL Image Encoder achieved an AMS of 73.52 with 665.3M parameters. Replacing it with CLIP improved AMS to 75.29 while reducing parameters to 427.6M. Siglip-2 increased accuracy further to 75.75 but required the largest parameter count at 881.5M. Our final choice, the DINO encoder, reached the highest AMS of 76.34 with only 304.4M parameters. This confirms that DINO offers the best balance, providing high-quality visual features for adaptive grounding while remaining computationally lightweight for our iSHIFT agent. We present more insight into the VPM in Supplementary.

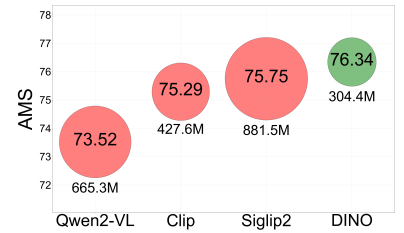


Figure 8. Performance of alternative visual encoders in the VPM. Circle size reflects encoder parameter count.

6. Conclusion

We introduced iSHIFT, a compact multimodal GUI agent that integrates implicit slow-fast control with latent reasoning and selective perception. By combining latent thinking tokens with an on-demand Visual Perception Module, iSHIFT allocates resources adaptively, engaging in deeper visual grounding only when necessary. This design removes external controllers and maintains high precision across diverse interface layouts. Extensive experiments on AITW, GUI Odyssey, Android Control, and GUIAct show that iSHIFT matches or surpasses much larger models, establishing a new efficiency-accuracy trade-off for GUI agents. These results highlight implicit adaptive reasoning as a powerful approach for building capable and resource-aware multimodal agents.

Acknowledgements

We gratefully acknowledge the support of IIT Bombay and IIT Hyderabad for this project. We also sincerely thank the anonymous reviewers, area chairs, and program chairs for their valuable feedback, which helped improve the quality and presentation of the paper.

References

- [1] Chongyang Bai, Xiaoxue Zang, Ying Xu, et al. Uibert: Learning generic multimodal representations for ui understanding. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, pages 1705–1712. International Joint Conferences on Artificial Intelligence Organization, 2021. Main Track. 2
- [2] Shuai Bai, Keqin Chen, Xuejing Liu, et al. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 6
- [3] Yuxiang Chai, Siyuan Huang, Yazhe Niu, et al. AMEX: Android multi-annotation expo dataset for mobile GUI agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 2138–2156, Vienna, Austria, 2025. Association for Computational Linguistics. 5
- [4] Wentong Chen, Junbo Cui, Jinyi Hu, et al. GUICourse: From general vision language model to versatile GUI agent. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21936–21959, Vienna, Austria, 2025. Association for Computational Linguistics. 5, 6
- [5] Kanzhi Cheng, Qiushi Sun, Yougang Chu, et al. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, Bangkok, Thailand, 2024. Association for Computational Linguistics. 1, 2, 5, 6
- [6] Konstantina Christakopoulou, Shibl Mourad, and Maja Matarić. Agents thinking fast and slow: A talker-reasoner architecture. *arXiv preprint arXiv:2410.08328*, 2024. 2
- [7] Xiangxiang Chu, Limeng Qiao, Xinyang Lin, et al. Mobilevlm: A fast, reproducible and strong vision language assistant for mobile devices. *arXiv preprint arXiv:2312.16886*, 2023. 2, 5
- [8] Xiang Deng, Yu Gu, Boyuan Zheng, et al. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, pages 28091–28114. Curran Associates, Inc., 2023. 1, 2
- [9] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, et al. Training large language models to reason in a continuous latent space, 2024. 3
- [10] Wenyi Hong, Weihang Wang, Qingsong Lv, et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14281–14290, 2024. 2, 5, 6
- [11] Zhiyuan Huang, Ziming Cheng, Junting Pan, et al. Spirit-sight agent: Advanced gui agent with one look. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pages 29490–29500, 2025. 2
- [12] Wei Li, William Bishop, Alice Li, et al. On the effects of data scale on ui control agents, 2024. 5, 6
- [13] Kevin Qinghong Lin, Linjie Li, Difei Gao, et al. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pages 19498–19508, 2025. 2, 5
- [14] Junpeng Liu, Yifan Song, Bill Yuchen Lin, et al. Visual-webbench: How far have multimodal LLMs evolved in web page understanding and grounding? In *First Conference on Language Modeling*, 2024. 1
- [15] Quanfeng Lu, Wenqi Shao, Zitao Liu, et al. Gui odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. *arXiv preprint arXiv:2406.08451*, 2024. 1, 5, 6
- [16] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. Omniparser for pure vision based gui agent, 2024. 1, 5
- [17] Xinbei Ma, Zhuosheng Zhang, and Hai Zhao. Comprehensive cognitive llm agent for smartphone gui automation. *arXiv preprint arXiv:2402.11941*, 2024. 5
- [18] Dang Nguyen, Jian Chen, Yu Wang, et al. Gui agents: A survey. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22522–22538, 2025. 1
- [19] OpenAI, :, Aaron Hurst, Adam Lerer, Adam P. Goucher, et al. Gpt-4o system card, 2024. 5, 6
- [20] Maxime Oquab, Timothée Darcet, Théo Moutakanni, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. Featured Certification. 3, 5
- [21] Yujia Qin, Yining Ye, Junjie Fang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025. 1
- [22] Abdur Rahman, Rajat Chawla, Muskaan Kumar, et al. V-zen: Efficient gui understanding and precise grounding with a novel multimodal llm. *arXiv preprint arXiv:2405.15341*, 2024. 2, 5
- [23] Christopher Rawles, Alice Li, Daniel Rodriguez, et al. Androidinthewild: A large-scale dataset for android device control. In *Advances in Neural Information Processing Systems*, pages 59708–59728. Curran Associates, Inc., 2023. 1, 5, 6
- [24] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. In *The Thirteenth International Conference on Learning Representations*, 2025. 1
- [25] Peter Shaw, Mandar Joshi, James Cohan, et al. From pixels to ui actions: Learning to follow instructions via graphical user interfaces. In *Advances in Neural Information Processing Systems*, pages 34354–34370. Curran Associates, Inc., 2023. 2
- [26] Huawei Shen, Chang Liu, Gengluo Li, et al. Falcon-ui: Understanding gui before following user instructions, 2024. 2
- [27] Guangyan Sun, Mingyu Jin, Zhenting Wang, et al. Visual agents as fast and slow thinkers. In *The Thirteenth International Conference on Learning Representations*, 2025. 2
- [28] Fei Tang, Yongliang Shen, Hang Zhang, et al. Think twice, click once: Enhancing gui grounding via fast and slow systems, 2025. 2

- [29] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models, 2023. [5](#)
- [30] Junyang Wang, Haiyang Xu, Jiabo Ye, et al. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158*, 2024. [3](#), [5](#)
- [31] Peng Wang, Shuai Bai, Sinan Tan, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution, 2024. [4](#), [5](#)
- [32] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, pages 24824–24837. Curran Associates, Inc., 2022. [3](#)
- [33] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, et al. OS-ATLAS: Foundation action model for generalist GUI agents. In *The Thirteenth International Conference on Learning Representations*, 2025. [3](#), [6](#)
- [34] Yiheng Xu, Zekun Wang, Junli Wang, et al. Aguis: Unified pure vision agents for autonomous GUI interaction. In *Forty-second International Conference on Machine Learning*, 2025. [3](#), [6](#)
- [35] An Yan, Zhengyuan Yang, Wanrong Zhu, et al. Gpt-4v in wonderland: Large multimodal models for zero-shot smartphone gui navigation. *arXiv preprint arXiv:2311.07562*, 2023. [1](#), [2](#)
- [36] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. [3](#)
- [37] Keen You, Haotian Zhang, Eldon Schoop, et al. Ferret-ui: Grounded mobile ui understanding with multimodal llms, 2024. [2](#)
- [38] Runpeng Yu, Xinyin Ma, and Xinchao Wang. Introducing visual perception token into multimodal large language model, 2025. [3](#)
- [39] Zhuosheng Zhan and Aston Zhang. You only look at screens: Multimodal chain-of-action agents. *arXiv preprint arXiv:2309.11436*, 2023. [5](#)
- [40] Bofei Zhang, Zirui Shang, Zhi Gao, et al. Tongui: Building generalized gui agents by learning from multimodal web tutorials. *arXiv preprint arXiv:2504.12679*, 2025. [2](#), [5](#), [6](#)
- [41] Zhuosheng Zhang and Aston Zhang. You only look at screens: Multimodal chain-of-action agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3132–3149, Bangkok, Thailand, 2024. Association for Computational Linguistics. [5](#)
- [42] Boyuan Zheng, Boyu Gou, Jihyung Kil, et al. Gpt-4v(ision) is a generalist web agent, if grounded. In *Forty-first International Conference on Machine Learning*, 2024. [1](#)